

MWORKS.Syslab DSP系统工具箱

苏州同元软控信息技术有限公司

2025年8月21日



TONGYUAN
Software and Control

@苏州同元软控信息技术有限公司
版权所有，侵权必究

目录

1. DSP系统工具箱功能概况

2. 信号生成、处理和分析

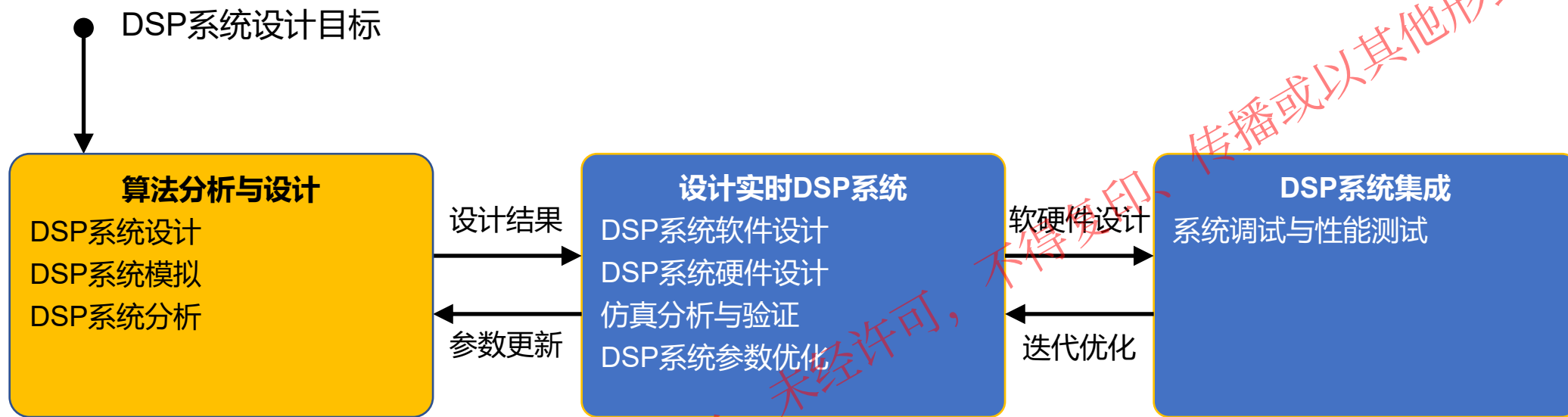
3. 滤波器设计与分析

4. 滤波器系统对象实现

5. 变换与谱分析

6. 统计学与线性代数

1. DSP系统工具箱功能概述



DSP系统工具箱为DSP系统**算法分析与设计**阶段提供计算支撑

1. DSP系统工具箱功能概述

DSP系统工具箱提供了多种算法和功能用于设计、仿真和分析数字信号处理系统。

1 信号生成、处理和分析

- 信号创建
- 信号导入导出
- 显示和管理信号

2 滤波器设计与分析

- FIR, IIR数字滤波器设计
- 模拟滤波器
- 滤波器分析
- 滤波器频带转换

3 滤波器系统对象实现

- 单速率滤波器
- 多级多速率滤波器
- 自适应滤波器

4 变换与谱分析

- 傅立叶变换
- 谱分析, 线性预测

5 统计学与线性代数

- 测量和统计

目录

1. DSP系统工具箱功能概况

2. 信号生成、处理和分析

3. 滤波器设计与分析

4. 滤波器系统对象实现

5. 变换与谱分析

6. 统计学与线性代数

2.1 信号生成、处理和分析 – 信号生成

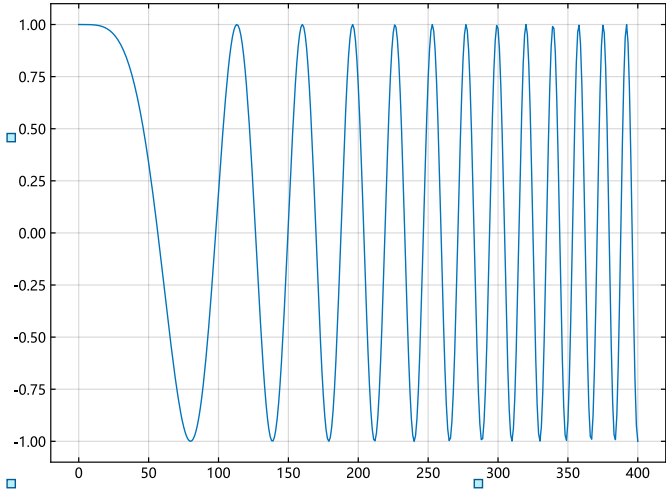
目前支持以下的函数

函数名	简介
dsp_Chrip	生成扫频余弦（线性调频）信号
dsp_ColoredNoise	产生有色噪声信号
dsp_NCO	生成实数或复数的正弦信号
dsp_SignalSource	从工作空间导入变量
dsp_SineWave	生成离散正弦波

示例： Syslab中生成一个0-25HZ的扫频余弦信号并绘图.

```
#生成一个0-25HZ的扫频余弦信号并绘图
chi = dsp_Chrip(
    InitialFrequency=0, TargetFrequency=25, SampleRate=400, SamplesPerFrame=400
) #定义扫频余弦对象
y = step(chi)#生成信号
figure()
plot(y)
grid("on")
```

输出结果：



2.1 信号生成、处理和分析 – 信号处理和分析

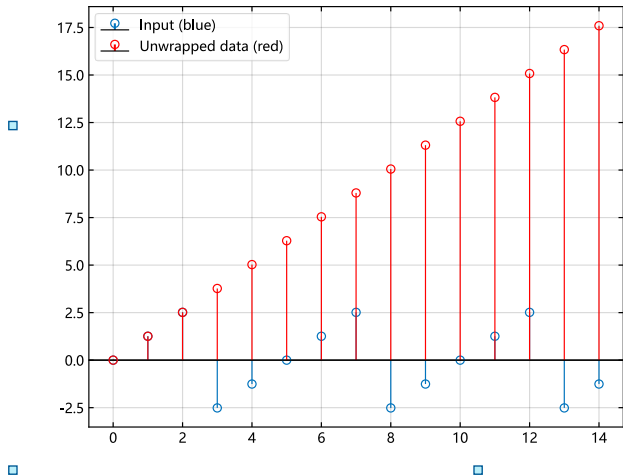
目前支持以下的函数

函数名	简介
dsp_FarrowRateConverter	具有任意转换系数的多项式采样率转换器
dsp_DCBlocker	阻止输入信号中的直流分量(偏移)
dsp_PhaseExtractor	提取复数输入的展开相位
dsp_PhaseUnwrapper	信号相位解卷绕
dsp_ZeroCrossingDetector	检测过零点
dsp_Delay	固定采样延迟输入信号

示例： Syslab中对信号相位解卷绕：

```
#信号相位解卷绕
phUnwrap = dsp_PhaseUnwrapper()#定义相位解卷绕的对象
p = [0 2/5 4/5 -4/5 -2/5 0 2/5 4/5 -4/5 -2/5 0 2/5 4/5 -4/5 -2/5] * pi#定义相位数据
y = step(phUnwrap, p)#相位解卷绕
figure()
stem(p)
hold("on")
stem(y; linefmt="r", markerfmt="ro")
legend(["Input (blue)", "Unwrapped data (red)"])
grid("on")
```

输出结果：



2.1 信号生成、处理和分析 – 信号处理和分析

示例： Syslab中对信号进行延迟：

```
#对信号进行延迟
del = dsp_Delay(; Length=4) #定义信号延迟对象，并
设置延迟长度为4
data = [1:10 11:20] #定义数据
y1 = step(del, data) #进行信号延迟
```

输出：

```
10×2 Matrix{Int64}:
 0  0
 0  0
 0  0
 0  0
 1 11
 2 12
 3 13
 4 14
 5 15
 6 16
```

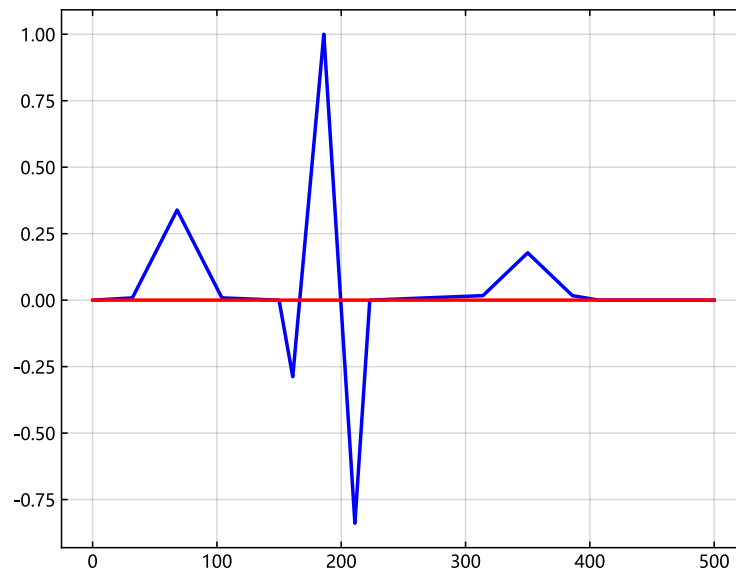
示例： Syslab中对信号进行过零点检测：

```
#信号进行过零点检测
EcgData = ecg(500) #生成ecg数据
zcd = dsp_ZeroCrossingDetector() #定义过零点检测
对象
numZeroCross = step(zcd, EcgData) #过零点检测
figure()
plot(1:500, EcgData, "b", [0 500], [0 0], "r";
linewidth=2)
grid("on")
```

显示过零点数量numZeroCross：

```
julia> numZeroCross
1×1 Matrix{Int64}:
 4
```

绘制信号图：



目录

1. DSP系统工具箱功能概况

2. 信号生成、处理和分析

3. 滤波器设计与分析

4. 滤波器系统对象实现

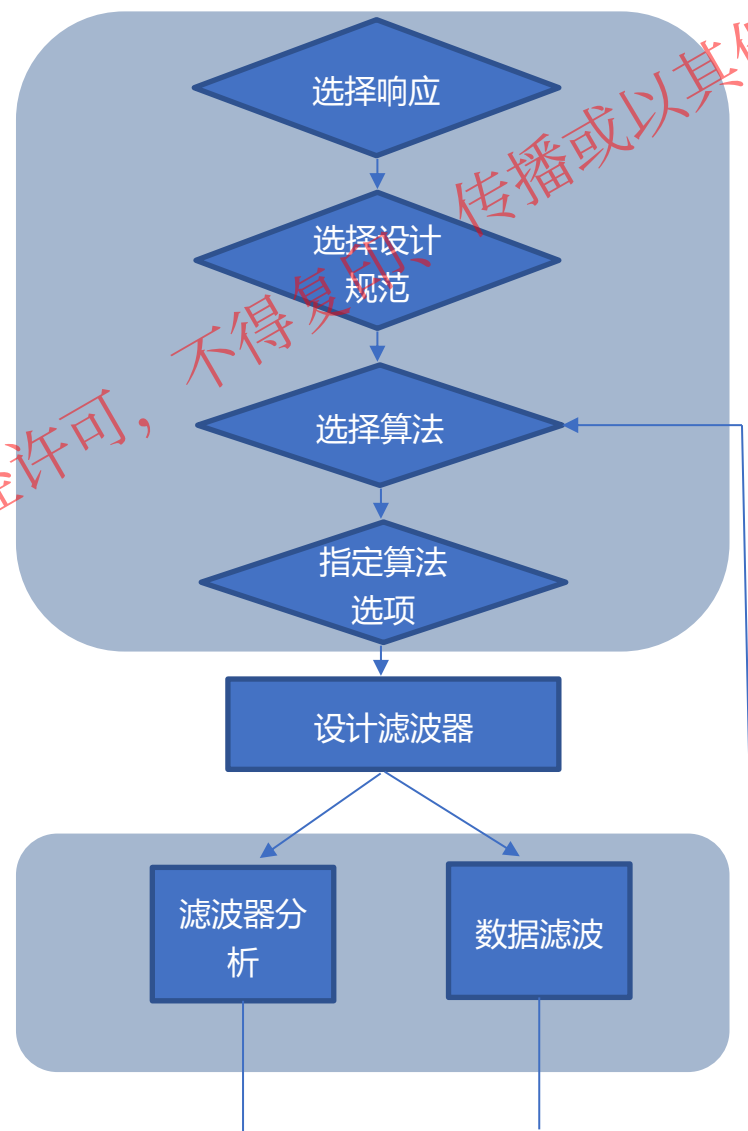
5. 变换与谱分析

6. 统计学与线性代数

3.1 滤波器设计与分析 - 使用规格化参数设计一个滤波器

使用fdesign系列函数设计一个滤波器：

- ❑1.选择一个可用的滤波器响应（低通，高通，带通，带阻等）；
- ❑2.在该响应类型中选择一个设计规范，得到设计规范对象；
- ❑3.在该规范对象可用的设计算法中选择一种算法；
- ❑4.可设置算法的自定义选项；
- ❑5.使用design函数设计滤波器；
- ❑6.可使用滤波器分析函数对创建的滤波器进行特性分析；
- ❑7.可利用设计和优化的滤波器进行滤波操作。



设计规范对象

根据分析和滤波结果优化设计

3.1 滤波器设计与分析 - 使用规格化参数设计一个滤波器

一. 选择一个可用的滤波器响应（低通，高通，带通，带阻等）

滤波器响应的标准格式为 `fdesign_response`, 目前支持的响应类型可在帮助文档的滤波器设计板块找到。

示例： Syslab 中生成一个默认低通规格对象：

```
#生成一个默认低通规格对象  
d = fdesign_lowpass()
```

输出：

```
lowpass - 属性:  
  Response: Lowpass  
  Specification: "Fp,Fst,Ap,Ast"  
  Description: 4×1 Tuple  
  NormalizedFrequency: true  
  Fpass: 0.45  
  Fstop: 0.55  
  Apass: 1.0  
  Astop: 60.0
```

示例： Syslab 中生成一个默认任意幅度响应规格对象：

```
#生成一个默认任意幅度响应规格对象  
d = fdesign_arbmag()
```

输出：

```
arbmag - 属性:  
  Response: Arbitrary Magnitude  
  Specification: "N,F,A"  
  Description: 3×1 Tuple  
  NormalizedFrequency: true  
  FilterOrder: 30  
  Frequencies: 1×49 Matrix{Float64}  
  Amplitudes: 1×49 Matrix{Float64}
```

3.1 滤波器设计与分析 - 使用规格化参数设计一个滤波器

二.在该响应类型中选择一个设计规范，得到设计规范对象

设计规范是给定滤波器的设计参数字符串，其也是规范对象的一个属性。

当选定滤波器响应之后，会有许多不同的可用设计规范，其中每个设计规范都包含有不同与目标响应相关的参数组合。

目前可在滤波器响应的帮助文档中查询到可用的设计规范。

示例： Syslab中生成一个指定阶数和3dB点的高通规格对象：

```
#生成一个指定阶数和3dB点的高通规格对象  
d = fdesign_highpass("N,F3dB", 7, 0.6)
```

输出：

```
highpass - 属性:  
    Response: Highpass  
    Specification: "N,F3dB"  
    Description: 2×1 Tuple  
    NormalizedFrequency: true  
    FilterOrder: 7  
    F3dB: 0.6
```

- "Fst,Fp,Ast,Ap" (默认规格)
- "N,F3dB"
- "N,F3dB,Ap"
- "N,F3dB,Ast"
- "N,F3dB,Ast,Ap"
- "N,F3dB,Fp"
- "N,Fc"
- "N,Fc,Ast,Ap"
- "N,Fp,Ap"
- "N,Fp,Ast,Ap"
- "N,Fst,Ast"
- "N,Fst,Ast,Ap"
- "N,Fst,F3dB"
- "N,Fst,Fp"
- "N,Fst,Fp,Ap"
- "N,Fst,Fp,Ast"
- "Nb,Na,Fst,Fp"

示例： Syslab中生成一个自定义参数的默认带阻规格对象：

```
#生成一个自定义参数的默认带阻规格对象  
d = fdesign_bandstop(0.3, 0.4, 0.6, 0.7, 0.5, 60, 1)
```

输出：

```
bandstop - 属性:  
    Response: bandstop  
    Specification: "Fp1,Fst1,Fst2,Fp2,Ap1,Ast,Ap2"  
    Description: 7×1 Tuple  
    NormalizedFrequency: true  
    Fpass1: 0.3  
    Fstop1: 0.4  
    Fstop2: 0.6  
    Fpass2: 0.7  
    Apass1: 0.5  
    Astop: 60.0  
    Apass2: 1.0
```

3.1 滤波器设计与分析 - 使用规格化参数设计一个滤波器

三.在该规范对象可用的设计算法中选择一种算法

算法取决于选择的滤波器响应和设计规范。也就是说，对于相同的低通滤波器来说，不同的设计规范会对应不同的设计算法。可以使用designmethod函数来查询指定规范对象可用的设计算法，会以字符串向量的形式返回。

示例： Syslab中查询高通规格对象可用的设计算法

```
#查询高通规格对象可用的设计算法
d = fdesign_highpass("N,F3dB", 7, 0.6)
designmethods(d)
```

输出：

```
2-element Vector{String}:
"butter"
"maxflat"
```

示例： Syslab中查询默认低通规格对象可用的设计算法：

```
#查询默认低通规格对象可用的设计算法
d = fdesign_lowpass()
designmethods(d)
```

输出：

```
8-element Vector{String}:
"butter"
"cheby1"
"cheby2"
"ellip"
"equiripple"
"ifir"
"kaiserwin"
"multistage"
```

3.1 滤波器设计与分析 - 使用规格化参数设计一个滤波器

四. 设置算法的自定义选项

算法的自定义选项取决于选择的滤波器响应，设计规范以及设计算法。可以使用`help(d, "method")`来获取自定义选项的详细信息（目前`help`仅支持低通滤波器的默认设计规范），也可以使用`designoptions(d, "method")`来获取当前规格对象的`method`算法支持自定义的选项、默认值和取值范围。

示例： Syslab中查询低通滤波器默认设计规范的butter算法的自定义选项

```
#查询低通滤波器默认设计规范的butter算法的自定义选项
designSpecs = fdesign_lowpass()
designoptions(designSpecs, "butter")
```

输出：

```
Dict{Any, Any} with 10 entries:
"DefaultSOSScaleNorm" => ""
"DefaultSystemObject" => false
"DefaultSOSScaleOpts" => fdopts_sosscaling("auto", 2, "none", "wrap", "unit", "Not used", nothing, nothing)
"SystemObject" => "Bool"
"SOSScaleNorm" => "String"
"SOSScaleOpts" => "fdopts_sosscaling"
"DefaultMatchExactly" => "stopband"
"FilterStructure" => ["df1sos", "df2sos", "df1tsos", "df2tsos", "cascadeallpass", "cascadewdfallpass"]
"MatchExactly" => ["passband", "stopband"]
"DefaultFilterStructure" => "df2sos"
```

3.1 滤波器设计与分析 - 使用规格化参数设计一个滤波器

五.使用design函数设计滤波器

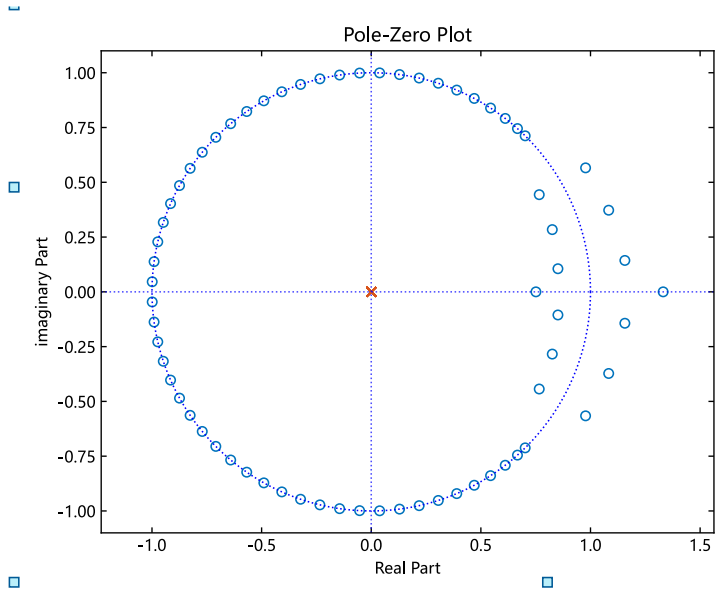
使用design函数设计滤波器。

调用方式	说明
<code>filt = design(designSpecs, "Systemobject", true)</code>	使用滤波器规范对象 <code>designSpecs</code> 来生成滤波器系统对象 <code>filt</code> 。当您不提供设计方法作为输入参数时，设计使用默认设计方法。使用 <code>designmethods</code> 函数查看滤波器设计规范对象的默认设计方法。
<code>filt = design(designSpecs, method, "Systemobject", true)</code>	使用 <code>method</code> 指定的设计方法。 <code>method</code> 必须是 <code>designmethods</code> 返回的选项之一。
<code>filt = design(designSpecs, method, PARAM, VALUE, ..., "Systemobject", true)</code>	指定设计方法选项。使用 <code>designoptions(designSpecs,method)</code> 查看可供选择的可用设计方法选项列表。

示例： Syslab中设计一个低通滤波器并查看其零极点图

```
#设计一个低通滤波器并查看其零极点图
#生成低通滤波器规格对象，其中通带边缘频率为0.2π rad/sample，阻带边缘频率为0.25π
rad/sample，通带纹波为 0.5 dB，阻带衰减为 40 dB
designSpecs = fdesign_lowpass("Fp,Fst,Ap,Ast", 0.2, 0.25, 0.5, 40)
filt1 = design(designSpecs, "SystemObject", true)
fvtool(filt1; Analysis="polezero");
```

输出结果：



3.1 滤波器设计与分析 - 使用规格化参数设计一个滤波器

六.使用滤波器分析函数对创建的滤波器进行特性分析

可以对设计的滤波器对象应用滤波器分析函数，目前支持以下的分析函数：

函数名	说明
freqz	离散时间滤波器的频率响应
freqrespopts	为频率响应估计创建一个选项对象
freqrespst	通过滤波的频率响应估计
grpdelay	离散时间滤波器的群时延响应
impz	离散时间滤波器的脉冲响应
impzlength	离散滤波器的脉冲响应长度
measure	测量滤波器系统对象的频率响应特性
noisepsdopts	为输出噪声 PSD 计算创建一个选项对象
noisepsd	由于舍入噪声引起的滤波器输出的功率谱密度
phasedelay	离散时间滤波器系统对象的相位延迟响应
phasez	离散时间滤波器的相位响应系统对象
stepz	离散时间滤波器的阶跃响应
zerophase	离散时间滤波器的零相位响应
zplane	离散时间滤波器的Z平面零极点图



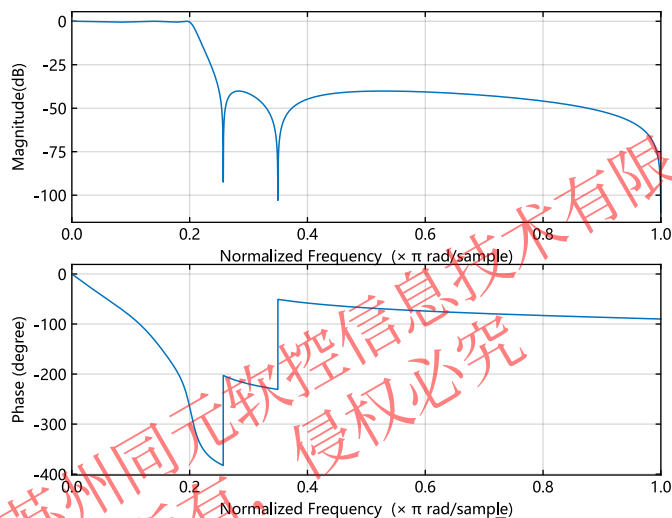
3.1 滤波器设计与分析 - 使用规格化参数设计一个滤波器

六.使用滤波器分析函数对创建的滤波器进行特性分析

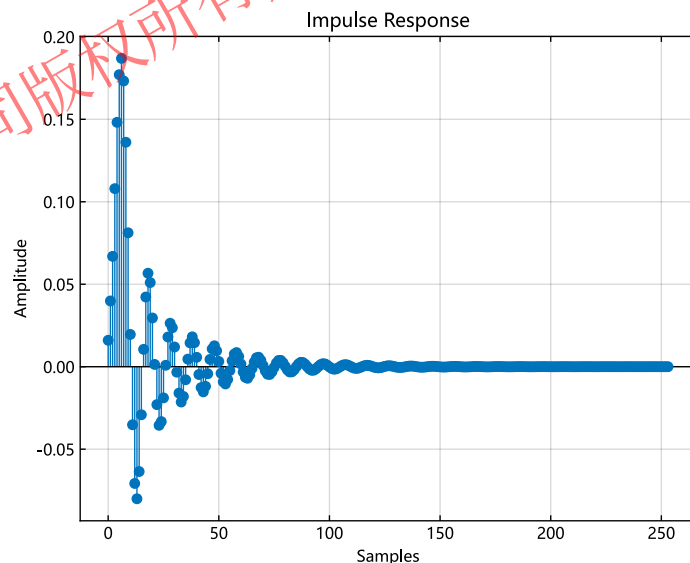
示例： Syslab中设计一个椭圆低通滤波器并查看其频率响应，脉冲响应，相位延迟

```
#设计一个椭圆低通滤波器并查看其频率响应
#生成低通滤波器规格对象，其中通带边缘频率为 $0.2\pi$  rad/sample，阻带边缘频率为 $0.25\pi$  rad/sample，通带纹波为 0.5 dB，阻带衰减为 40 dB
designSpecs = fdesign_lowpass("Fp,Fst,Ap,Ast", 0.2, 0.25, 0.5, 40)
filt = design(designSpecs, "ellip", "MatchExactly", "passband", "SOSScaleNorm", "linf",
"SystemObject", true) #生成指定为通带匹配，L-infinity 范数缩放
freqz(filt; plotfig=true)
impz(filt; plotfig=true)
figure()
phasedelay(filt; plotfig=true)
```

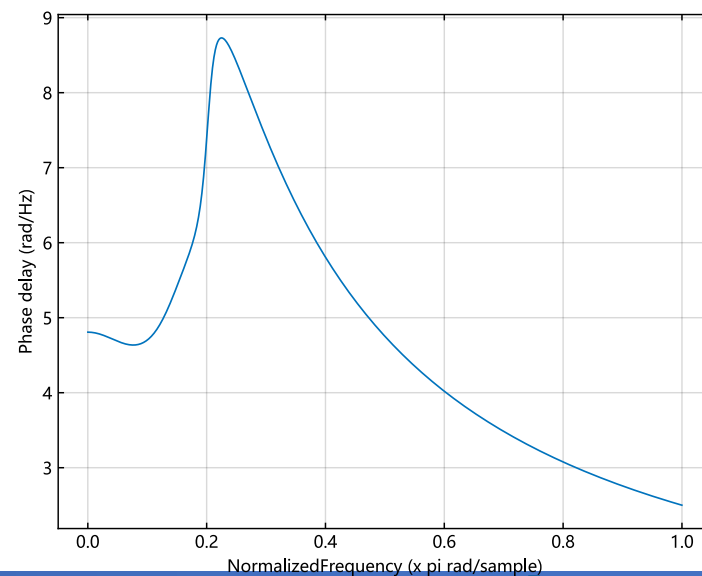
频率响应



脉冲响应



相位延迟



目录

1. DSP系统工具箱功能概况

2. 信号生成、处理和分析

3. 滤波器设计与分析

4. 滤波器系统对象实现

5. 变换与谱分析

6. 统计学与线性代数

4.1 滤波器系统对象实现 – 基于RLS自适应滤波器的FIR系统估计

自适应滤波器：在很多信号处理系统中，并没有信号的先验统计特性，不能使用某一固定参数的滤波器来处理，比如信道均衡、回声消除以及其他因素之间的系统模型等，均采用了调整系数的滤波器，称为自适应滤波器。这样的滤波器结合了允许滤波器系数适应于信号统计特性的算法。

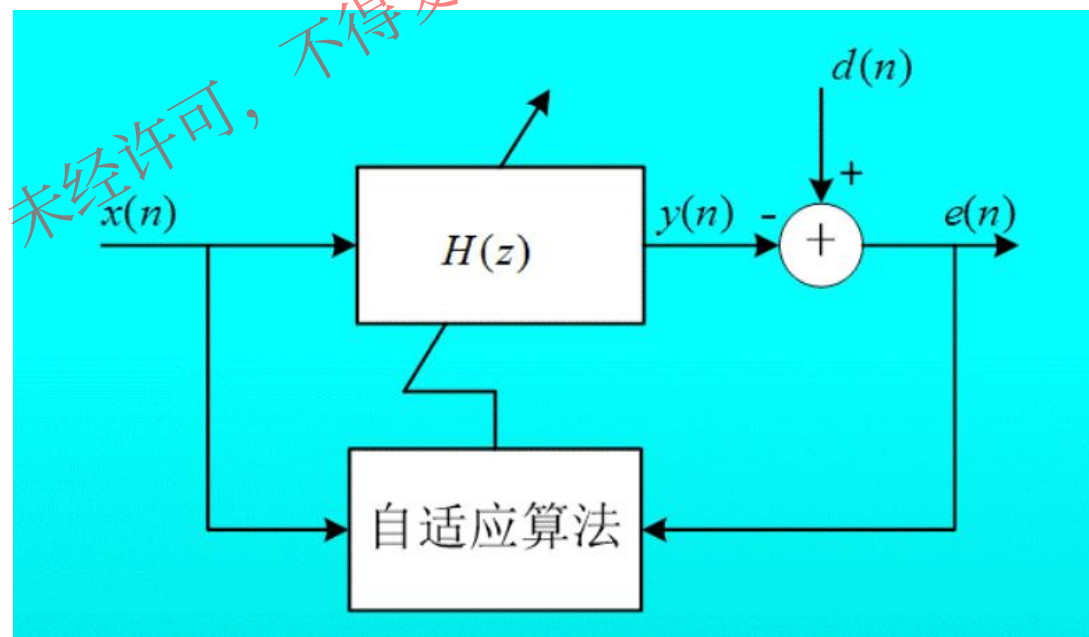
自适应滤波器的原理框图如下图所示，输入信号 $x(n)$ 通过参数可调数字滤波器后产生输出信号 $y(n)$ ，将其与期望信号 $d(n)$ 进行比较，形成误差信号 $e(n)$ ，通过自适应算法对滤波器参数进行调整，最终使 $e(n)$ 的均方值最小。

按照算法分类可以分为最小均方（LMS）和最小二乘(RLS)。

RLS自适应滤波器：是一种基于最小二乘法的自适应滤波算法，它的基本原理是利用现有的观测数据来估计所需的滤波器系数，最小化预测误差的均方差来优化滤波效果。

其基本算法流程为：

- 1.初始化滤波器系数向量。
- 2.将观测数据输入滤波器，得到预测输出。
- 3.计算预测输出与实际输出之间的误差。
- 4.根据误差和一定的学习率，更新滤波器系数向量。
- 5.重复上述步骤，不断调整滤波器系数，直到达到满意的滤波效果。



通用自适应滤波器原理图

4.1 滤波器系统对象实现 – 基于RLS自适应滤波器的FIR系统估计

使用递归最小二乘 (RLS) 滤波器来识别使用低通 FIR 滤波器建模的未知系统。比较未知系统和估计系统的频率响应。

初始化目标系统和期望信号：创建代表要识别的系统的 `dsp_FIRFilter` 对象。将信号 `x` 传递给 FIR 滤波器。期望信号 `d` 是未知系统 (FIR 滤波器) 的输出和加性噪声信号 `n` 之和。

```
rng = MT19937ar(1234)
firf = dsp_FIRFilter(, Numerator=fir1(10, 0.25))#生成未知系统对象
x = randn(rng, 1000)
n = 0.01 * randn(rng, 1000)
d = step(firf, x) + n#使用未知系统对象对数据进行滤波并加噪声
```

定义RLS自适应滤波器并进行系统估计：创建 `dsp_RLSFilter` 对象以创建 RLS 滤波器。将滤波器的长度设置为 11，将遗忘因子设置为 0.98。这里使用Sliding-window RLS算法，可通过设置对象的Method覆盖切换算法。将初级输入信号 `x` 和所需信号 `d` 传递给 RLS 滤波器。自适应滤波器的输出 `y` 是收敛到所需信号 `d` 的信号，从而使两个信号之间的误差 `e` 最小化。

```
rls = dsp_RLSFilter(, Length=11, ForgettingFactor=0.98)#定义
RLS滤波器对象，指定滤波器阶数和遗忘因子

rls.Method = "Sliding-window RLS"#设置RLS滤波器对象的算法
# rls.Method = "Householder RLS"
# rls.Method = "Householder sliding-window RLS"
# rls.Method = "QR-decomposition RLS"
y, e = step(rls, x, d)#使用RLS滤波器对象进行系统估计
w = rls._Coefficients
```

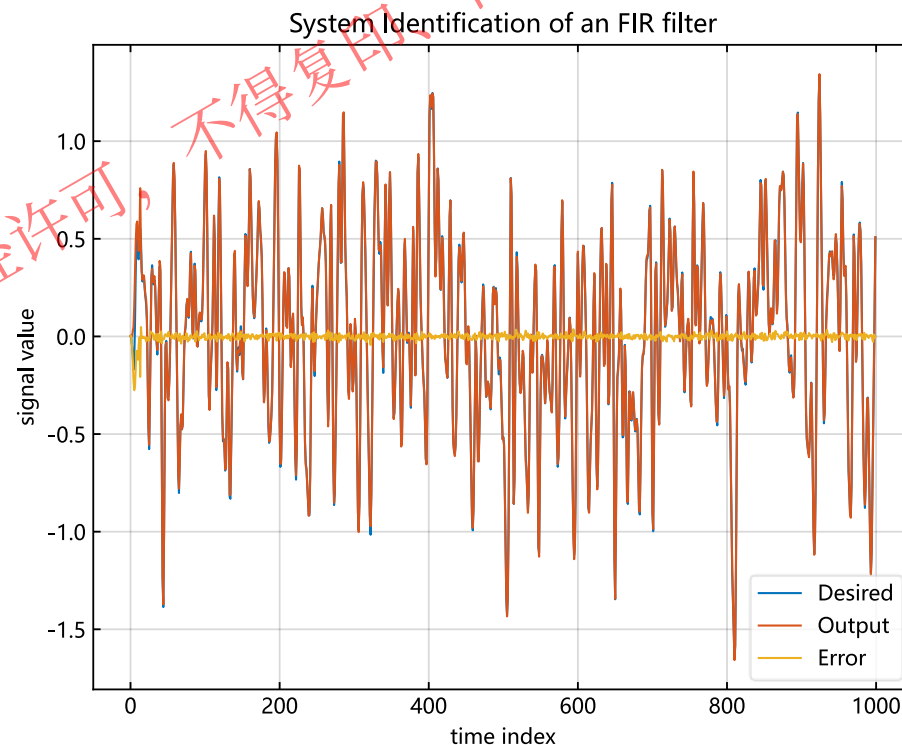
4.1 滤波器系统对象实现 – 基于RLS自适应滤波器的FIR系统估计

使用递归最小二乘 (RLS) 滤波器来识别使用低通 FIR 滤波器建模的未知系统。比较未知系统和估计系统的频率响应。

绘制信号对比图形：从图像上可以看出信号与期望信号匹配，两者之间的误差接近于零。

```
figure()
plot(d)#绘制期望输出
hold("on")
plot(y)#绘制RLS滤波器输出
plot(e)#绘制误差
title("System Identification of an FIR filter")
legend(["Desired", "Output", "Error"])
xlabel("time index")
ylabel("signal value")
grid("on")
```

输出结果：



4.1 滤波器系统对象实现 – 基于RLS自适应滤波器的FIR系统估计

使用递归最小二乘 (RLS) 滤波器来识别使用低通 FIR 滤波器建模的未知系统。比较未知系统和估计系统的频率响应。

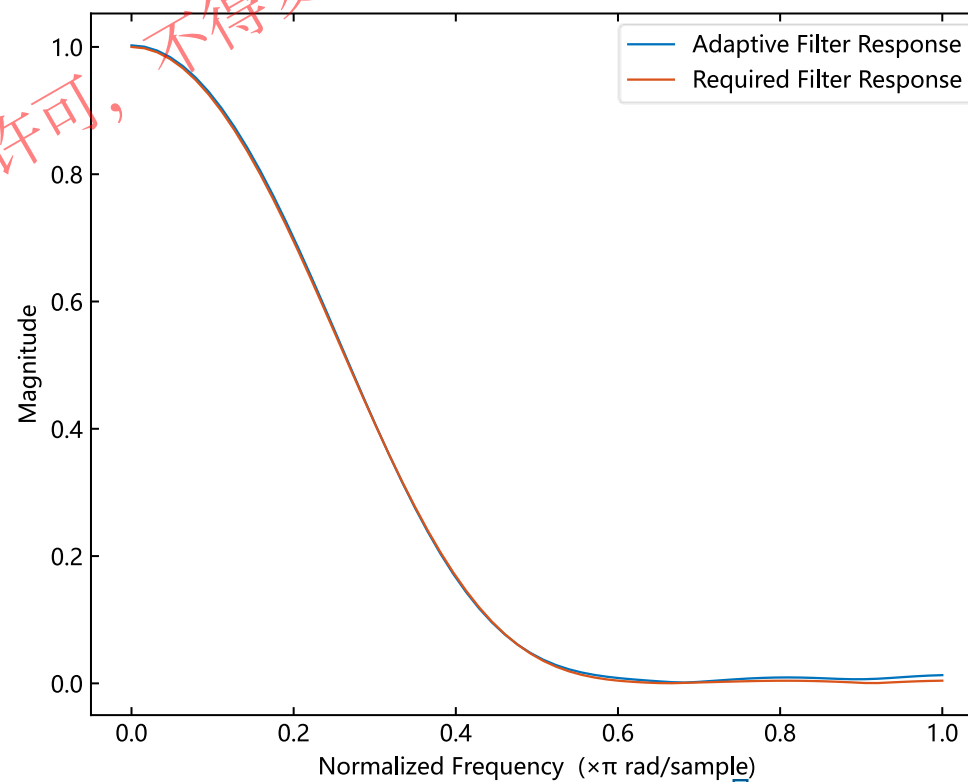
绘制频率响应图形：该图显示了自适应滤波器响应对FIR滤波器响应的收敛性。

```
h, = freqz(rls._Coefficients, 1, 64)#求滤波器的频率响应
H = abs.(h);
h1, = freqz(firf.Numerator, 1, 64)
H1 = abs.(h1);

wf = range(0, 1, 64);

plot(wf, H, wf, H1);#绘制滤波器的频率响应图
xlabel("Normalized Frequency ( $\times\pi$  rad/sample)");
ylabel("Magnitude");
legend("Adaptive Filter Response", "Required Filter Response");
```

输出结果：



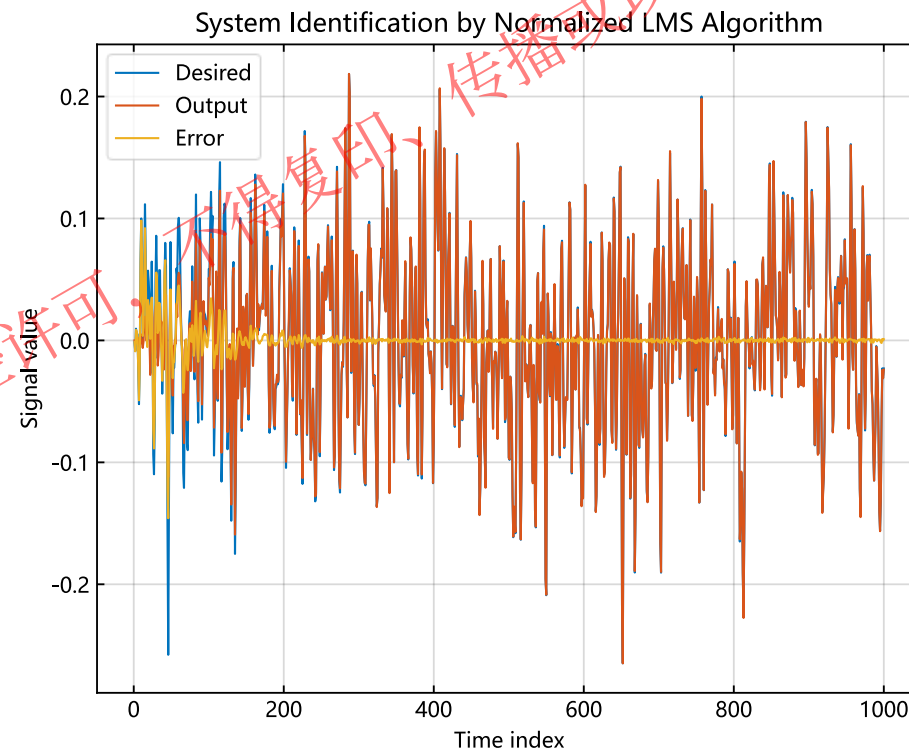
4.1 滤波器系统对象实现 – 基于LMS自适应滤波器的FIR系统估计

使用最小均方 (LMS) 滤波器来识别使用低通 FIR 滤波器建模的未知系统。比较未知系统和估计系统的频率响应。

```
firf = dsp_FIRFilter(  
    Numerator=[  
        0.0164196271386271,  
        0.103050485715060,  
        -0.0631687353150344,  
        -0.0906631463336251,  
        0.0467158877053708,  
        0.313883690306561,  
        0.452608500318065,  
        0.313883690306561,  
        0.0467158877053708,  
        -0.0906631463336251,  
        -0.0631687353150344,  
        0.103050485715060,  
        0.0164196271386271,  
    ],  
)  
rng = MT19937ar(1234)  
x = 0.1 * randn(rng, 1000)  
n = 0.001 * randn(rng, 1000)  
d = step(firf, x) + n
```

```
# Adaptive Filter  
mu = 0.2  
lms = dsp_LMSFilter(Length=13,  
    StepSize=mu, Method="Normalized  
    LMS")  
y, e, w = step(lms, x, d)  
figure()  
plot([1:1000;], [d y e])  
title("System Identification by  
    Normalized LMS Algorithm")  
legend(["Desired", "Output", "Error"])  
xlabel("Time index")  
ylabel("Signal value")  
grid("on")
```

输出结果:



自适应滤波器的输出 y 是收敛到所需信号 d 的信号，从而使两个信号之间的误差 e 最小化。如图可以看到随着信号长度的增加误差越来越小。

目录

1. DSP系统工具箱功能概况

2. 信号生成、处理和分析

3. 滤波器设计与分析

4. 滤波器系统对象实现

5. 变换与谱分析

6. 统计学与线性代数

5.1 变换与谱分析

目前提供傅立叶变换，频谱分析的相关函数

信号的频域表示揭示了难以在时域分析中展示的重要特性。DSP 系统工具箱中的dsp_FFT和dsp_IFFT系统对象和模块可以将信号在时域和频域相互转化。使用dsp_SpectrumEstimator计算信号的频谱估计。

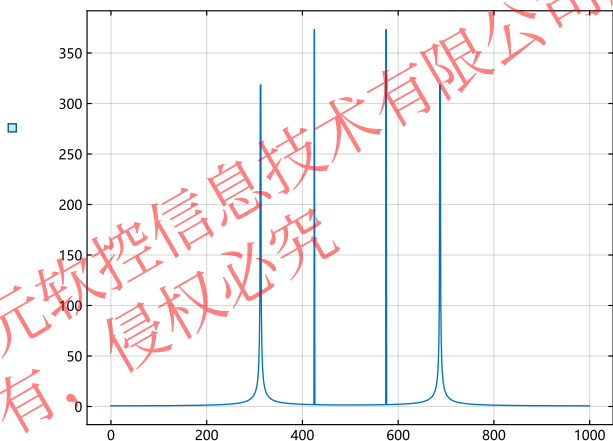
函数名	简介
dsp_FFT	离散傅里叶变换
dsp_IFFT	离散傅里叶逆变换
dsp_STFT	短时 FFT
dsp_ISTFT	短时逆 FFT
dsp_SpectrumEstimator	估计功率谱或功率密度谱
dsp_CrossSpectrumEstimator	估计交叉频谱密度
dsp_TransferFunctionEstimator	估计传递函数

5.1 变换与谱分析 – 离散傅里叶变换及逆变换

示例： Syslab中离散傅里叶变换：

```
#离散傅里叶变换
ft = dsp_FFT(; FFTLength=1000)#定义FFT对象
Fs = 800
L = 1000
t = [0:(L - 1)] / Fs
data = sin.(2 * pi * 250 * t) + 0.75 * cos.(2 * pi * 340 * t)
#生成信号

outdata = step(ft, data)#FFT变换
figure()
plot(abs.(outdata))#绘制幅度谱
grid()
```



示例： Syslab中离散傅里叶逆变换：

```
#离散傅里叶逆变换
ift = dsp_IFFT(; FFTLength=1000)#定义IFFT对象
ft = dsp_FFT(; FFTLength=1000)#定义FFT对象
Fs = 800
L = 1000
t = [0:(L - 1)] / Fs
data = sin.(2 * pi * 250 * t) + 0.75 * cos.(2 * pi * 340 * t)#生成信号
out = step(ft, data)#FFT变换
outdata = step(ift, out)#IFFT变换
maximum(real.(outdata) .- data)#计算误差
```

输出：

8.881784197001252e-16

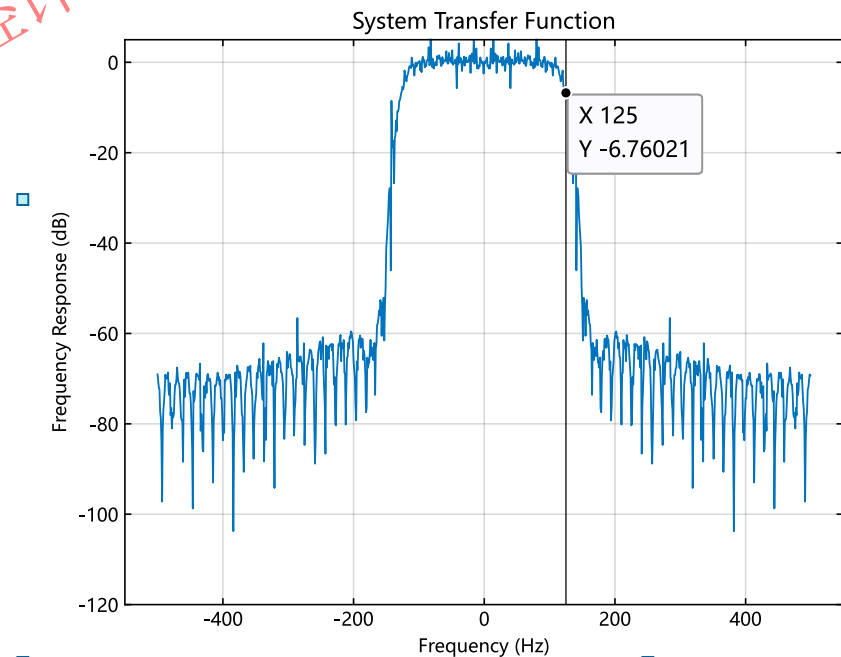
逆变换后信号与源信号误差为e-15级，傅里叶逆变换结果正确

5.1 变换与谱分析 – 目标系统的传递函数估计

传递函数是描述系统输入和输出之间关系的数学函数。它可以用于分析系统的稳定性、频率响应和时域响应等特性。传递函数估计是一种通过实验数据来估计系统传递函数的方法。具体计算方法为：计算目标系统的输入信号和输出信号的频谱（使用Welch的平均修正周期图方法），通过计算输入信号的输出信号的相干性得到目标系统的频率响应即为传递函数。

```
rng = MT19937ar(1234)#使用随机数种子
sine = dsp_SineWave(; Frequency=100, SampleRate=1000,
SamplesPerFrame=1000)#生成正弦波
tfe = dsp_TransferFunctionEstimator(; FrequencyRange="centered")#创建传
递函数估计对象
plotter = dsp_ArrayPlot(;
    PlotType="Line",
    XOffset=-500,
    YLimits=[-120 5],
    Title="System Transfer Function",
    YLabel="Frequency Response (dB)",
    XLabel="Frequency (Hz)",
)#创建绘图对象
LP = dsp_FIRFilter(; Numerator=fir1(64, 1 / 4))#创建FIR滤波器（作为未知系
统）即截止频率为125HZ
x = step(sine) + 0.05 * randn(rng, 1000, 1)#生成输入信号
y = step(LP, x)#通过未知系统
Txy, = step(tfe, x, y)#使用输入信号估计传递函数
step(plotter, 20 * log10.(abs.(Txy)))
```

输出结果：



目录

1. DSP系统工具箱功能概况

2. 信号生成、处理和分析

3. 滤波器设计与分析

4. 滤波器系统对象实现

5. 变换与谱分析

6. 统计学与线性代数

6.1 统计学与线性代数

目前主要提供动态统计与测量相关函数

可以使用以下函数测量信号的移动统计数据 and 静止统计。移动统计是指随时间变化的流信号的统计。在计算移动统计量时有以下的两种方法：滑动窗口方法，当新数据进入时，指定长度的窗口在数据样本上逐样本移动。对象和块计算该窗口内数据的统计信息。指数加权法对数据样本赋予一组权重，并对加权后的数据进行处理。这些权重是根据数据的时间递归计算的。

函数名	简介
dsp_MovingAverage	动态均值
dsp_MovingMaximum	动态最大值
dsp_MovingMinimum	动态最小值
dsp_MovingRMS	动态均方根
dsp_MovingStandardDeviation	动态标准差
dsp_MovingVariance	动态方差
powermeter	测量电压信号的功率和对应的 CCDF
dsp_PeakToRMS	峰值均方根



6.1 统计学与线性代数 – 使用dsp_MovingAverage计算移动平均数

移动平均数：在统计学中是一种通过创建整个数据集中不同子集的一系列平均数来分析数据点的计算方法。它也是有限脉冲响应滤波器的一种。包括：滑动窗口移动平均、加权移动平均、指数加权移动平均以及累积移动平均。移动平均通常与时间序列数据一起使用，以消除短期波动，突出长期趋势或周期。**dsp_MovingAverage**支持滑动窗口移动平均和指数加权移动平均。可以通过其“**Method**”属性进行选择。

滑动窗口移动平均，使用以下公式递归计算移动平均：

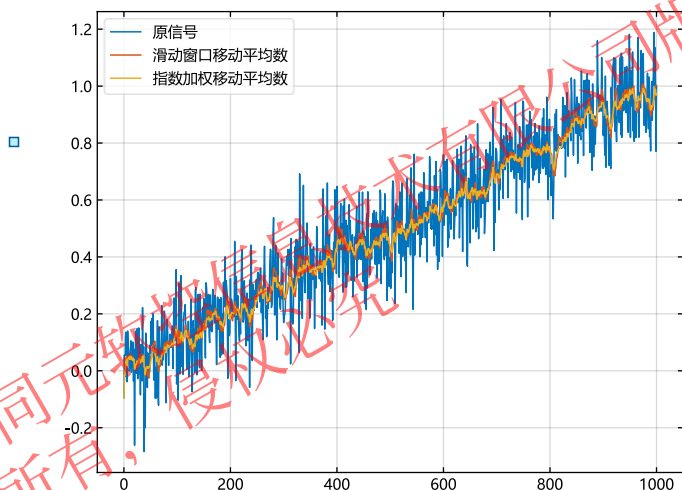
$$SMA_t(n) = \frac{1}{n} \sum_{i=t-n+1}^t x_i$$

其中， n 为窗口大小， $SMA_t(n)$ 表示 t 时刻的移动平均值。

指数加权移动平均中，使用以下公式递归计算移动：

$$EMA_t = \frac{x_t + (1-\alpha)x_{t-1} + (1-\alpha)^2 x_{t-2} + \dots + (1-\alpha)^t x_0}{1 + (1-\alpha) + (1-\alpha)^2 + \dots + (1-\alpha)^t}$$

其中， n 为窗口大小， $SMA_t(n)$ 表示 t 时刻的移动平均值。



示例： Syslab中计算移动平均数：

```
#计算移动平均数
rng = MT19937ar(1234)
mOV1 = dsp_MovingAverage(; WindowLength=10)#定义滑动窗口移动平均滤波器对象
t = [0:0.001:1];
ramp = t;
x = ramp + 0.1 * randn(rng, length(t), 1)#生成数据

y1 = step(mOV1, x)#使用滑动窗口移动平均滤波器对象进行滤波

mOV2 = dsp_MovingAverage(; Method="Exponential weighting", ForgettingFactor=0.9)#定义指数加权移动平均滤波器对象
y2 = step(mOV2, x)#使用指数加权移动平均滤波器对象进行滤波
figure()
plot(x)
hold("on")
plot(y1)
grid("on")
plot(y2)
legend("原信号", "滑动窗口移动平均数", "指数加权移动平均数");
```

建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

感谢聆听

传播或以其他方式使用

未经许可，不得复制或

@苏州同元软控信息技术有限公司
版权所有，侵权必究